

Sixdof Falcon ROS2 Package User Guide

Version 2

© 2017-2025 Six Degrees Space Ltd.
Legal disclaimer: All hardware and software (including firmware) included in the product (“Product”) is licensed to Customer pursuant to a nonexclusive, limited license, solely for its intended purpose. Customer may not copy, disclose, modify, translate or adapt the Product, and may not disassemble or reverse engineer the Product, or seek in any manner to discover, disclose or use any proprietary design (included but not limited to optics, algorithms and techniques) contained therein. SIXDOF AND ITS SUPPLIERS SHALL IN NO EVENT BE LIABLE TO CUSTOMER (OR ANY THIRD PARTY), WITH RESPECT TO ANY CLAIM RELATING TO THE PRODUCT, WHETHER BASED IN CONTRACT, TORT (INCLUDING NEGLIGENCE), STRICT LIABILITY OR OTHERWISE (EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGES OR IF SUCH DAMAGES ARE FORESEEABLE) FOR: LOST PROFITS OR INDIRECT, INCIDENTAL, SPECIAL, PUNITIVE OR CONSEQUENTIAL DAMAGES; OR ANY AMOUNTS EXCEEDING (IN THE AGGREGATE) THE PURCHASE PRICE FOR THE PRODUCT.

Contents

1 Introduction **3**

2 Prerequisites **4**

3 ROS2 Package Setup **5**
- 3.1 Environment Setup 5
- 3.2 Building the Package 5

4 Sixdof Falcon ROS2 Interface **6**
- 4.1 Topics 6
- 4.2 Services 6
- 4.3 Example 7

1 Introduction

The goal of this document is to explain how to build and use the Sixdof Falcon ROS2 Package, `falcon_pkg`. This package serves as a ROS2 wrapper for the SdsFalcon C++ Runtime Library, providing an interface for leveraging its powerful capabilities in robotics applications.

2 Prerequisites

Before proceeding, ensure that you have ROS2 installed and set up on your computer. If you do not already have it installed, follow the instructions in the ROS2 Installation Tutorial. Be sure to select the version that corresponds to the Ubuntu version installed on your machine. For instance:

- Use ROS2 Jazzy for Ubuntu 24.04.
- Use ROS2 Humble for Ubuntu 22.04

Once ROS2 is installed, verify the setup by running:

```
source /opt/ros/<ros2-distro>/setup.bash
```

In the command above, replace `<ros2-distro>` with the name of the ROS2 distribution you have installed, such as `jazzy`, `humble`, or any other version you are using.

For more information on environment setup and using ROS2, refer to the ROS2 Tutorials.

Ensure that the ROS2 workspace is properly sourced before continuing with the setup of `falcon_pkg`.

3 ROS2 Package Setup

This section will guide you through setting up the environment for and building the Sixdof Falcon ROS2 Package, `falcon_pkg`.

3.1 Environment Setup

The directory `ros2_falcon` will act as your ROS2 workspace for the Sixdof Falcon ROS2 Package, `falcon_pkg`. In `ros2_falcon/src/falcon_pkg`, create a directory named `lib`, and copy into it the SdsFalcon binary that matches your platform from the Sixdof Falcon software release. The resulting directory structure should look as follows:

```
ros2_falcon/
└── src/
    └── falcon_pkg/
        ├── src/                    # C++ code
        ├── include/                # Header files
        │   └── falcon_pkg/
        │       ├── SdsFalcon.h
        │       └── SdsShutterControl.h
        ├── lib/                    # Libray binaries
        │   └── libSdsFalcon.so
        ├── msg/                    # Custom messages
        ├── srv/                    # Custom services
        ├── launch/                 # Launch files
        ├── CMakeLists.txt          # Build configuration for C++
        └── package.xml             # Package metadata
```

3.2 Building the Package

To build the `falcon_pkg` and enable its ROS2 interface, follow these steps:

1. Open a new terminal and ensure that your ROS2 environment is sourced.
2. In the terminal, navigate to your ROS2 workspace. Using the example name, this would mean navigating to the `ros2_falcon` directory.
3. Build the `falcon_pkg` using `colcon` by running the following command:

   ```
   colcon build --symlink-install --packages-select falcon_pkg
   ```

4. Before using the package, you must source the workspace to include the compiled package in your environment:

   ```
   source install/setup.bash
   ```

5. Note that in each new terminal, you must source the workspace before using the package.

4 Sixdof Falcon ROS2 Interface

All the functionality from the SdsFalcon C++ Runtime Library is provided through the ROS2 interface by the `falcon_manager_node.hpp` and `falcon_manager_node.cpp` files, which can be found in the `falcon_pkg/src` directory. These files contain the setup for all services and topic publishers. The other files in this directory are example nodes, which demonstrate how to use the ROS2 interface for specific functionalities.

4.1 Topics

The following ROS2 topics are provided by the `falcon_pkg`, along with their corresponding message types:

- **heartbeat** - SdsFalcon health data `[falcon_pkg/msg/Heartbeat]`
- **status_message** - SdsFalcon status messages `[falcon_pkg/msg/StatusMessage]`
- **relative_beacon** - Relative Beacon tracking mode output data `[falcon_pkg/msg/PoseRelativeBeaconCollection]`
- **relative_angle** - Relative Angle tracking mode output data `[falcon_pkg/msg/PoseRelativeAngleCollection]`
- **sixdof** - Sixdof tracking mode output data `[falcon_pkg/msg/Pose6Dof]`
- **current_shutter** - Current shutter value data `[falcon_pkg/msg/CurrentShutter]`
- **field_of_view_report** - Report of which beacons are in the field of view of the sensor `[falcon_pkg/msg/FieldOfViewReport]`
- **dynamic_message** - Dynamic messages that are transmitted to the Sixdof sensor optically via one of the Sixdof smart driver boards `[falcon_pkg/msg/DynamicMessage]`

Each topic publishes data using a custom message type, which contains the same information as the output from SdsFalcon.

To view a complete list of the topics and their types in the terminal run: `ros2 topic list -t`

4.2 Services

The following ROS2 services are provided by the `falcon_pkg`, along with their corresponding service types:

- **get_version** - invoke this service to obtain the SdsFalcon C++ Runtime Library version being used `[falcon_pkg/srv/GetVersion]`
- **get_firmware_version** - invoke this service to obtain the firmware version that is on the connected sensor `[falcon_pkg/srv/GetFirmwareVersion]`
- **initialize_network** - invoke this service to initialize the network configuration and connect to the sensor `[falcon_pkg/srv/InitializeNetwork]`

- **initialize_network_advanced** - invoke this service to initialize the network configuration and connect to the sensor using advanced settings `[falcon_pkg/srv/InitializeNetworkAdvanced]`
- **initialize_relative_beacon** - invoke this service to initialize the configuration for Relative Beacon tracking mode `[falcon_pkg/srv/InitializeRelativeBeacon]`
- **initialize_relative_angle** - invoke this service to initialize the configuration for Relative Angle tracking mode `[falcon_pkg/srv/InitializeRelativeAngle]`
- **initialize_6dof** - invoke this service to initialize the configuration for Sixdof tracking mode `[falcon_pkg/srv/Initialize6Dof]`
- **set_tracking_mode** - invoke this service to set and start a tracking mode, it can only be called with a tracking mode that has already been initialized `[falcon_pkg/srv/SetTrackingMode]`
- **swap_map** - invoke this service to dynamically swap the map for Sixdof tracking mode, it can only be called after Initialize6Dof service has been called `[falcon_pkg/srv/SwapMap]`
- **get_gyro_calibration** - invoke this service to do gyro calibration and retrieve the results `[falcon_pkg/srv/GetGyroCalibration]`
- **load_shutter_plugin** - invoke this service to dynamically set the custom shutter control logic from a compiled plugin, a valid plugin is in the form of a shared library (.so on Linux) where the function `extern "C" IShutterControl* createShutterControll()` is implemented `[falcon_pkg/srv/LoadShutterPlugin]`
- **load_default_shutter_control** - invoke this service to dynamically return the shutter logic to the default `[falcon_pkg/srv/LoadDefaultShutterControl]`

To view a complete list of the services and their types in the terminal run: `ros2 service list -t`

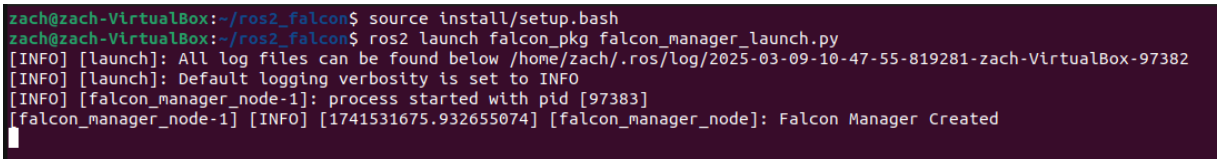
4.3 Example

This section contains an example of how to use the Sixdof Falcon ROS2 interface from the terminal. The example illustrates how to initialize the system, start a particular tracking mode, and receive data outputs in the terminal.

Before proceeding with the example, make sure that `falcon_pkg` is built. Additionally, **remember to source the package in each new terminal by running:** `source install/setup.bash`.

Use the following example instructions as a guide to interact with the Sixdof Falcon ROS2 package:

1. In a new terminal, launch the `falcon_manager_node`, which sets up all of the SdsFalcon services and publishers, by running: `ros2 launch falcon_pkg falcon_manager_launch.py`. The launch file can be found in the `falcon_pkg/launch` directory.
   Important: The first time you run `falcon_manager_node`, it may require `sudo` permissions to connect to the sensor. If you start the node with `sudo`, all other terminals running ROS functionality that interact with this node will also need `sudo` permissions.



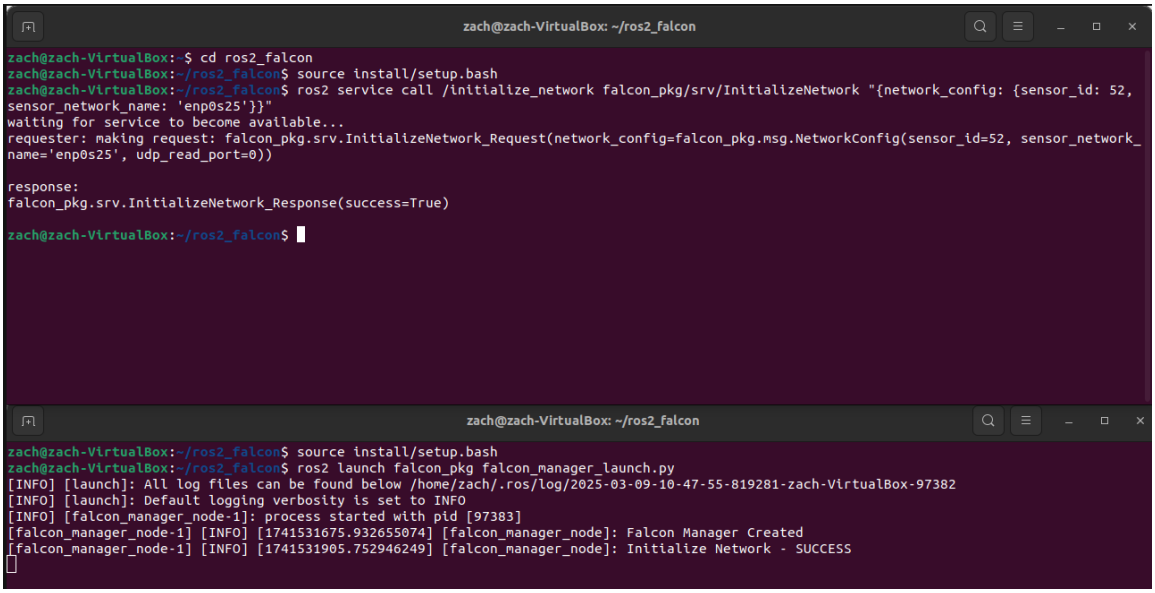
2. Similar to with the SdsFalcon C++ Runtime Library, it is important to subscribe to the topics that will output data before starting the tracking.

a) In a new terminal, run: `ros2 topic echo /sixdof`. Once the 6DOF tracking mode is started, this terminal will display a continuous stream of the positional data.

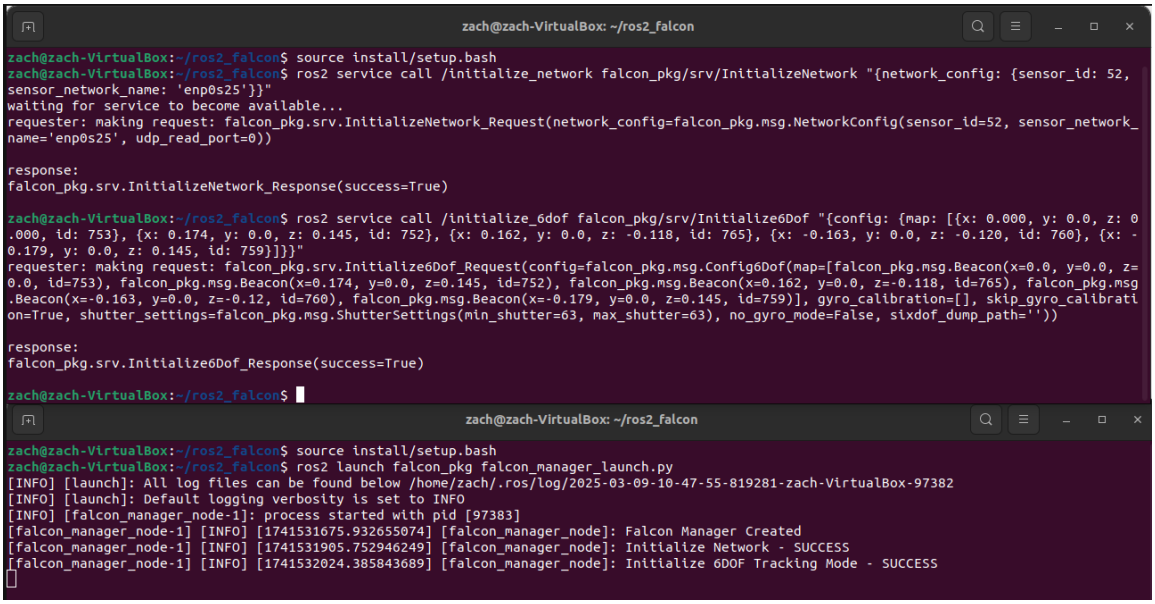
b) In another new terminal, run: `ros2 topic echo /status_message`. This terminal will show the outputted status messages from the system.

3. Just as with the SdsFalcon C++ Runtime Library, the network must be initialized first, followed by the initialization of the tracking mode, and only then the tracking mode can be set and started.

   a) In a new terminal, call the `initialize_network` service with the sensor's network configuration data. Replace the placeholder values with your actual data and run the command:
   `ros2 service call /intialize_network falcon_pkg/srv/InitializeNetwork "{network_config: {sensor_id: 52, sensor_network_name: 'eth0'}}"`

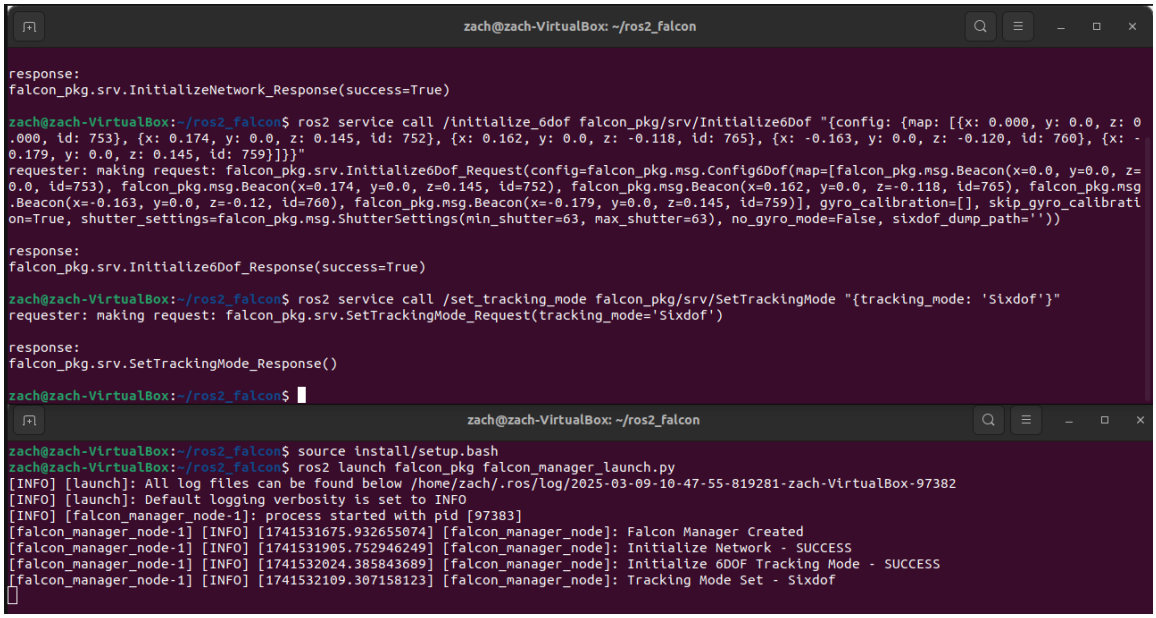


   b) In the same terminal, call the `initialize_6dof` service with the 6DOF tracking mode configuration data. Replace the placeholder values with your actual data and run the command:
   `ros2 service call /intialize_6dof falcon_pkg/srv/Initialize6Dof "{ config: { map: [ {x: 0.0, y: 0.0, z: 0.0, id: 1}, {x: 0.0, y: 0.0, z: 0.0, id: 2} ] } }"`



   c) In the same terminal, call the `set_tracking_mode` service to start the 6DOF Tracking Mode. Run: `ros2 service call /set_tracking_mode falcon_pkg/srv/SetTrackingMode`

```
"{tracking_mode: 'Sixdof'}"
```



4. After this setup, the 6DOF tracking should be running and outputting data in the two dedicated terminals:

